CC1352R Platformunda İkinci Firmware İmajının Yerleştirilmesi ve Boot Sürecinin İncelenmesi

BİL 304 İşletim Sistemleri Dönem Çalışması — 3. İş Parçacığı

Yasin Engin El-Hâris Halef

2026-01-01

İçindekiler

1 Özet 2

2 Giriş 2

3 Çalışmanın Kapsamı 3

4 Kullanılan Proje Yapısı 3

5 Firmware İmajlarının Boyut Analizi 4

6 CC1352R Flash ve RAM Yerleşimi 4

7 Boot Zinciri ve BIM Davranışı 6

8 CCFG Alanının Önemi 8

9 İkinci Firmware İçin Yerleşim ve Aktivasyon Stratejisi 9

10 Riskler ve Sınırlılıklar 12

11 Gelecek Çalışmalar 13

12 Sonuç 14

13 Ekler 15

13.1 Ek A — `old-firmware` Size Çıktısı 15

13.2 Ek B — `new-firmware` Size Çıktısı 15
13.3 Ek C — Section Analizi Çıktıları 15
13.4 Ek D — ELF Header Özetleri 16
13.5 Ek E — Ham Çıktı Dosyaları 16

Kaynakça **16**

1 Özet

Bu rapor, Texas Instruments CC1352R platformunda iki firmware imajının aynı flash bellek üzerinde güvenli biçimde konumlandırılmasını ve reset sonrasında hangi imajın çalıştırılacağını belirleyen boot sürecini incelemektedir. Çalışma, ağ üzerinden OTA paket aktarımı tasarlamaya değil, üçüncü iş parçacığı olan gerçek donanım yerleşimi ve boot uyarlamasına odaklanır. Bu kapsamda `old-firmware` persistent fallback imajı, `new-firmware` ise BIM tarafından öncelikli değerlendirilen user image olarak ele alınmıştır.

Derleme sonucunda elde edilen ELF, section, sembol ve boyut çıktıları ham halde `docs/report/outputs/` altında saklanmıştır. Ana raporda bu ham çıktıların doğrudan dökümü yerine, bunlardan üretilen karşılaştırmalı tablolar ve kısa teknik yorumlar kullanılmıştır. Sonuç olarak, yeni firmware imajının yalnızca flash belleğe yazılmasının yeterli olmadığı; geçerli OAD image header, doğru reset vector yerleşimi, slot sınırı kontrolü, BIM davranışı ve CCFG korumasının birlikte değerlendirilmesi gerektiği gösterilmiştir.

Anahtar Kelimeler: CC1352R, firmware güncelleme, BIM, OAD, CCFG, flash yerleşimi, ELF analizi, Contiki-NG, ARM Cortex-M.

2 Giriş

Gömülü sistemlerde firmware güncelleme işlemi, yalnızca yeni uygulama kodunun derlenmesi veya cihaza yazılması değildir. Mikrodenetleyici reset aldığında çalışmanın hangi adresten başlayacağı, uygulama imajının hangi başlık bilgilerini taşıdığı ve boot yöneticisinin bu imajı geçerli kabul edip etmediği sistemin güvenilirliğini doğrudan belirler. CC1352R gibi kaynak kısıtlı mikrodenetleyicilerde flash bellek aynı anda uygulama imajlarını, boot yöneticisini ve cihaz konfigürasyon alanını barındırdığı için bellek yerleşimi tasarımın merkezindedir (Instruments, t.y.-a, t.y.-b).

Bu çalışma, BİL 304 dönem çalışmasının üçüncü iş parçacığı için hazırlanmıştır. Proje genel olarak firmware güncelleme bağlamından doğmuş olsa da bu raporun ana konusu Cooja simülasyonu, ACK/NACK mekanizması veya ağ üzerinden veri aktarımı değildir. Burada incelenen problem, Contiki-NG ile derlenen iki uygulama imajının CC1352R üzerinde mevcut TI BIM/OAD boot modeline uygun biçimde yerleştirilmesidir (*Contiki-NG Documentation*, t.y.; *Contiki-NG SimpleLink Platform*, t.y.; *Boot Image Manager*, t.y.).

3 Çalışmanın Kapsamı

Çalışmada yeni bir bootloader yazılmamış ve TI BIM kaynak kodu değiştirilmemiştir. Bunun yerine firmware tarafındaki linker script, OAD image header ve flash slot adresleri mevcut BIM'in beklediği yapıya göre düzenlenmiştir. `old-firmware`, persistent fallback imajı olarak `0x00030000` başlangıç adresine; `new-firmware`, user image olarak `0x00000000` başlangıç adresine yerleştirilmiştir.

Bu raporda üç temel soruya cevap aranmıştır. Birincisi, firmware imajlarının kod, veri ve RAM bölümleri bakımından ne kadar yer kapladığıdır. İkincisi, bu imajların CC1352R flash haritasında hangi slotlara yerleştiğidir. Üçüncüsü, reset sonrasında hangi imajın çalıştırılacağına uygulama kodunun değil, BIM ve boot zincirinin nasıl karar verdiğidir.

4 Kullanılan Proje Yapısı

Projede iki uygulama kaynak dosyası, iki OAD header kaynağı ve iki linker script birlikte kullanılır. Bu ayrım, aynı kod tabanından farklı boot rollerine sahip iki ayrı firmware imajı üretmeyi sağlar (Engin ve Halef 2026).

Tablo 1. Proje dosyaları ve görevleri

Dosya	Görevi
`old-firmware.c`	Persistent fallback uygulamasıdır; seri log üzerinden eski imajın çalıştığını gösterir.
`new-firmware.c`	User/update uygulamasıdır; BIM'in yeni imaja geçtiğini seri log üzerinden gösterir.
`oad_layout.h`	Flash slot adreslerini, slot boyutlarını, entry adreslerini ve sürümleri tanımlar.
`oad_hdr_old.c`	`old-firmware` için `PERSISTENT_APP` tipinde OAD image header üretir.
`oad_hdr.c`	`new-firmware` için `APPSTACKLIB` tipinde OAD image header üretir.
`old-firmware.ld`	Eski imajın header ve reset vector adreslerini persistent fallback slotuna taşır.
`new-firmware.ld`	Yeni imajın header ve reset vector adreslerini page 0 user slotuna taşır.
`Makefile`	İmajları ayrı derler, doğru header dosyasını seçer ve HEX/BIN çıktıları üretir.

Derleme çıktıları `build/simplelink/sensortag/cc1352r1/` altında, cihaza yazılacak HEX/BIN dosyaları ise `upload/` altında oluşur. Rapor kanıtı olarak kullanılan terminal çıktıları ayrıca `docs/report/outputs/` klasöründe saklanmıştır.

5 Firmware İmajlarının Boyut Analizi

Bu bölümde `old-firmware` ve `new-firmware` imajlarının derleme sonrası bellek kullanımları incelenmiştir. Amaç, iki firmware imajının flash ve RAM üzerindeki etkisini sayısal olarak görmek ve ikinci firmware imajının CC1352R flash belleğinde hangi koşullarda saklanabileceğini değerlendirmektir. Bu analizde derleme çıktılarından elde edilen `.text`, `.data` ve `.bss` değerleri kullanılmıştır.

Tablo 2. `old-firmware` ve `new-firmware` bellek kullanım karşılaştırması

Firmware İmajı	`.text`	`.data`	`.bss`	Toplam	Teknik Anlamı
`old-firmware`	72672 byte	1452 byte	14520 byte	88644 byte	Aktif çalışan ana firmware imajı
`new-firmware`	72276 byte	1452 byte	14520 byte	88248 byte	Flash üzerinde aday/staging imaj olarak saklanacak firmware

`.text` alanı çalıştırılabilir makine kodunu ve genellikle flash bellekte yer alan program bölümünü temsil eder. `.data` bölümü başlangıç değeri olan global veya statik değişkenleri ifade eder; bu bölümün ilk değerleri flash üzerinde tutulur, çalışma sırasında RAM'e kopyalanır. `.bss` bölümü ise başlangıçta sıfırlanan değişkenlerden oluşur ve doğrudan RAM kullanımını etkiler. Bu nedenle firmware imajının yalnızca dosya boyutuna bakmak yeterli değildir; kodun flash üzerindeki kapladığı alan ile çalışma zamanında RAM üzerinde oluşturduğu yük ayrı ayrı değerlendirilmelidir.

6 CC1352R Flash ve RAM Yerleşimi

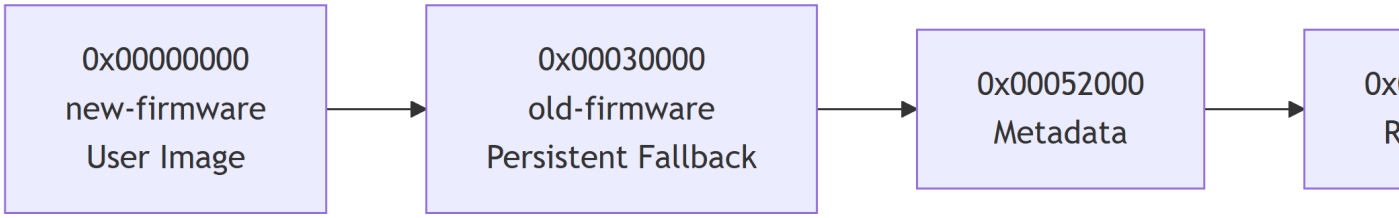
Bu bölümde CC1352R platformu için önerilen flash bellek yerleşimi incelenmiştir. Amaç, persistent fallback olarak tutulan `old-firmware` imajını korurken `new-firmware` imajını ayrı bir user/update bölgesinde saklayabilmektir. Bu yaklaşımda yeni firmware doğrudan fallback alanının üzerine yazılmaz. Bunun yerine BIM tarafından öncelikli değerlendirilen ayrı bir alanda tutulur, doğrulama bilgileri metadata bölgesinde saklanabilir ve reset sonrasında boot karar mekanizması tarafından değerlendirilir.

Tablo 5. CC1352R flash yerleşimi

Bölge	Başlangıç Adresi	Boyut	Kullanım Amacı	Teknik Not
User image / yeni imaj alanı	`0x00000000`	`0x00030000`	`new-firmware` imajının user/update imajı olarak konumlandığı alan	BIM reset sonrası önce bu alanı değerlendirir
Persistent fallback alanı	`0x00030000`	`0x00022000`	`old-firmware` imajının fallback olarak saklandığı alan	Yeni imaj yoksa veya geçersizse bu imaj çalıştırılır
Metadata alanı	`0x00052000`	`0x00002000`	Sürüm, boyut, CRC ve geçerlilik bilgisinin tutulabileceği alan	Boot kararı için kullanılabilir
Recovery alanı	`0x00054000`	`0x00002000`	Kurtarma veya fallback bilgileri için ayrılmış alan	Hatalı güncellemede geri dönüş için kullanılabilir
BIM + CCFG alanı	`0x00056000`	`0x00002000`	Boot yöneticisi ve cihaz konfigürasyon alanı	Kesinlikle korunmalıdır

Bu yerleşim stratejisinde flash bellek, user image, persistent fallback, metadata, recovery ve BIM/CCFG bölgeleri olarak mantıksal parçalara ayrılmıştır. Page 0 user image alanında güncelleme sonrası çalıştırılması hedeflenen `new-firmware` bulunur. `old-firmware` ise doğrudan bu alanın üzerine yazılmak yerine persistent fallback bölgesinde korunur. Böylece yeni imaj bozuk, eksik veya uyumsuz olsa bile mevcut fallback firmware cihazı tekrar çalıştırabilecek durumda kalır.

Metadata alanı, yeni imajın sürüm numarası, boyutu, CRC değeri ve geçerlilik durumu gibi bilgilerin tutulması için kullanılabilir. Boot karar mekanizması, reset sonrasında bu bilgileri kontrol ederek yeni imajın çalıştırılıp çalıştırılmayacağına karar verebilir. CCFG alanı ise cihazın düşük seviye konfigürasyon bilgilerini içerdiği için normal uygulama veya staging alanı gibi kullanılmamalıdır.



Bu bellek haritası, ikinci firmware imajının fallback uygulama alanından ayrı tutulduğunu göstermektedir. Bu ayrım, güvenli güncelleme tasarımı açısından önemlidir. Çünkü çalışan veya kurtarma için saklanan firmware'in üzerine doğrudan yazmak, işlem yarıda kesildiğinde cihazın açılmamasına neden olabilir. Ayrı user/update alanı kullanmak ise yeni imaj doğrulanana kadar fallback sisteminin korunmasını sağlar.

Tablo 6. Derleme çıktısına göre old/new firmware flash planı

Firmware İmajı	Slot Aralığı	Kullanılan Aralık	Raw BIN Boyutu	Boş Kalan Alan
`old-firmware`	`0x00030000 - 0x00051FFF`	`0x00030000 - 0x00042253`	74324 byte	64940 byte
`new-firmware`	`0x00000000 - 0x0002FFFF`	`0x00000000 - 0x000120C7`	73928 byte	122680 byte

RAM tarafında `old-firmware` ve `new-firmware` aynı uygulama düzenini kullanmaktadır. `.data`, `.bss`, `vtable_ram` ve `.stack` section'ları `0x20000000` tabanlı SRAM bölgesinde yer alır. Bu durum flash slotlarının farklı olmasına rağmen çalışan uygulamanın RAM modelinin aynı kaldığını gösterir.

7 Boot Zinciri ve BIM Davranışı

CC1352R platformunda firmware güncelleme süreci yalnızca yeni imajın flash belleğe yazılmasıyla tamamlanmış sayılmaz. Bir firmware imajının çalıştırılabilmesi için reset sonrasında işlemcinin doğru başlangıç adresine yönlendirilmesi, imajın geçerli olduğunun anlaşılması ve hatalı imaj durumunda güvenli geri dönüş yolunun bulunması gerekir. Bu nedenle ikinci firmware imajının flash bellekte veri veya aday imaj olarak tutulması ile bu imajın boot edilebilir hale gelmesi farklı konulardır.

Bu çalışmada `old-firmware`, cihaz üzerinde persistent fallback olarak saklanan güvenli geri dönüş imajı şeklinde değerlendirilmiştir. `new-firmware` ise fallback alanının üzerine yazılmamış, ayrı bir user/update alanında saklanan aday imaj olarak ele alınmıştır. Bu yaklaşımda yeni imajın çalıştırılması için reset sonrasında bir boot karar mekanizmasının devreye girmesi gerekir. CC1352R tarafında bu rol, BIM veya benzeri bir bootloader yapısı ile ilişkilendirilebilir (*Boot Image Manager*, t.y.; *Over-the-Air Download Overview for CC13xx and CC26xx*, t.y.).

Yukarıdaki boot karar akışı, önerilen reset sonrası seçim sürecini göstermektedir. Reset sonrasında ilk kontrol uygulama seviyesinde değil, boot aşamasında gerçekleşmelidir. Bu nedenle `new-firmware` dosyasının flash bellekte bulunması tek başına yeterli değildir. Boot mekanizması bu imajın başlangıç adresini, boyutunu, sürümünü ve bütünlük bilgisini kontrol edebilmelidir. Yeni imaj geçerli değilse sistemin `old-firmware` ile devam etmesi gerekir. Bu davranış, hatalı firmware nedeniyle cihazın tamamen kullanılamaz hale gelmesini önler.

Boot Aşaması	Görevi	Bu Projedeki Karşılığı
Reset	Cihazın başlangıç sürecini başlatır	Güç verilmesi veya yazılımsal reset sonrası ilk aşama
ROM boot kodu	Donanımın temel başlangıç sürecini yürütür	Cihazın düşük seviye boot davranışı

Boot Aşaması	Görevi	Bu Projedeki Karşılığı
BIM / Bootloader	Hangi imajın çalıştırılacağına karar verir	`old-firmware` veya `new-firmware` seçimi
Metadata kontrolü	Aday imajın geçerliliğini denetler	Sürüm, boyut, CRC ve imaj tipi kontrolü
Fallback	Hatalı imaj durumunda güvenli geri dönüş sağlar	`old-firmware` ile çalışmaya devam etme

Bu yapıdaki en kritik nokta, boot kararının uygulama çalıştıktan sonra değil, uygulama başlamadan önce verilmesidir. Çalışan bir firmware'in kendi üzerine yazılması riskli bir işlemdir; işlem yarıda kesilirse cihazın açılmama ihtimali oluşur. Bu nedenle güvenli firmware güncelleme tasarımında yeni imaj önce ayrı bir user/staging alanına yazılmalı, ardından bütünlük kontrolünden geçirilmeli ve en son boot mekanizması tarafından aktif hale getirilmelidir.

Bu çalışmada tam anlamıyla üretim seviyesinde bir firmware değiştirme sistemi gerçeklenmemiştir. Bunun yerine, CC1352R üzerinde ikinci firmware imajının nasıl konumlandırılabileceği, bu imajın boot süreciyle nasıl ilişkilendirilebileceği ve neden BIM/bootloader desteğine ihtiyaç duyulduğu analiz edilmiştir. Bu projede `new-firmware` için OAD image header `0x00000000` adresine, reset vector ise `0x00000100` adresine yerleştirilmiştir. `old-firmware` için karşılık gelen adresler `0x00030000` ve `0x00030100` değerleridir. Bu ayrım, BIM'in page 0 user image ve persistent fallback imajını farklı rollerle değerlendirebilmesini sağlar.

8 CCFG Alanının Önemi

CCFG, CC1352R platformunda cihazın düşük seviye konfigürasyon bilgilerinin tutulduğu kritik flash bölgelerinden biridir. Bu alan normal uygulama verisi gibi değerlendirilmemelidir. Çünkü CCFG bölgesi; boot davranışı, cihaz başlangıç ayarları, debug erişimi, flash koruma ayarları ve bazı donanımsal yapılandırmalar üzerinde etkili olabilir. Bu nedenle firmware güncelleme veya UniFlash ile yükleme işlemleri sırasında CCFG alanının yanlışlıkla silinmesi, cihazın beklenen şekilde başlamamasına veya programlama/debug sürecinde erişim problemleri oluşmasına yol açabilir (*CC13xx/CC26xx Bootloader and Customer Configuration*, t.y.; *UniFlash*, t.y.).

Bu çalışmada CCFG alanı, `old-firmware` veya `new-firmware` için kullanılacak sıradan bir saklama alanı olarak ele alınmamıştır. Aksine, korunması gereken özel bir konfigürasyon bölgesi olarak değerlendirilmiştir. Flash yerleşiminde yeni firmware imajı, metadata alanı veya recovery alanı planlanırken bu bölgelerin CCFG ile çakışmaması temel bir tasarım koşulu olarak kabul edilmiştir.

CCFG alanının kritik olmasının temel nedeni, boot zincirinin yalnızca uygulama kodundan ibaret olmamasıdır. Reset sonrasında cihazın nasıl davranacağı; ROM boot kodu, BIM/bootloader yapısı, uygulama imajı ve cihaz konfigürasyon alanlarıyla birlikte belirlenir. Dolayısıyla CCFG alanında oluşacak

bir bozulma, yalnızca uygulama seviyesinde bir hata oluşturmaz; cihazın başlangıç akışını doğrudan etkileyebilir.

Bu nedenle firmware güncelleme tasarımında üç temel güvenlik ilkesi izlenmelidir:

1. CCFG alanı uygulama veya staging verisi için kullanılmamalıdır.
2. Flash erase/write işlemleri sayfa sınırları dikkate alınarak yapılmalıdır.
3. UniFlash ile yükleme yapılırken CCFG bölgesinin yanlışlıkla silinmediği kontrol edilmelidir.

Özellikle ikinci firmware imajının flash üzerinde ayrı bir user/staging alanında saklandığı bu çalışmada, CCFG alanının korunması güvenli boot davranışı açısından zorunludur. Yeni imajın geçersiz olması durumunda sistemin `old-firmware` ile çalışmaya devam edebilmesi, yalnızca uygulama kodunun sağlam kalmasına değil, aynı zamanda boot zincirinde kullanılan konfigürasyon alanlarının da korunmasına bağlıdır.

Risk	Olası Sonuç	Alınması Gereken Önlem
CCFG alanının yanlışlıkla silinmesi	Cihazın beklenen şekilde başlamaması	Flash yerleşiminde CCFG ayrı ve korumalı bölge olarak tanımlanmalıdır
Yeni firmware'in CCFG ile çakışması	Boot veya debug davranışında bozulma	Linker ve layout dosyalarında adres sınırları kontrol edilmelidir
Tüm flash'ın bilinçsizce silinmesi	BIM, CCFG veya çalışan imajın kaybı	UniFlash işlemlerinde seçili erase yöntemi dikkatle kullanılmalıdır
Metadata/recovery alanının CCFG'ye taşması	Boot karar bilgilerinde veya cihaz ayarlarında hata	Boyut kontrolleri build sonrası doğrulama betikleriyle yapılmalıdır

Sonuç olarak CCFG alanı, firmware güncelleme sürecinde doğrudan üzerine veri yazılabilecek bir bellek alanı değildir. Bu alanın korunması, hem cihazın yeniden başlatılabilirliği hem de debug/programlama erişiminin sürdürülebilirliği açısından önemlidir. Bu nedenle önerilen bellek yerleşiminde CCFG bölgesi en sonda ayrı bir kritik alan olarak bırakılmış ve ikinci firmware imajı için kullanılmamıştır.

9 İkinci Firmware İçin Yerleşim ve Aktivasyon Stratejisi

Bu çalışmada ikinci firmware imajı, doğrudan aktif uygulama alanının üzerine yazılacak bir imaj olarak değil, flash bellekte ayrı bir bölgede tutulacak aday imaj olarak değerlendirilmiştir. Bunun temel nedeni, çalışan firmware'in üzerine doğrudan yazma işleminin güvenli olmamasıdır. Flash silme veya yazma

işlemi sırasında güç kesintisi, hatalı imaj, adres çakışması veya eksik veri gibi bir durum oluşursa cihazın mevcut çalışan uygulaması da bozulabilir. Bu nedenle yeni firmware önce ayrı bir staging alanına yerleştirilmeli, ardından doğrulanmalı ve ancak bundan sonra boot sürecinde aktif hale getirilmelidir.

Önerilen yapıda `old-firmware`, cihazın normalde çalıştığı ana uygulama imajıdır. `new-firmware` ise flash üzerinde ayrılmış ikinci bir bölgede veri/adayı imaj olarak saklanır. Bu aday imaj, tek başına çalıştırılmaz; önce metadata bilgileriyle birlikte doğrulanır. Metadata alanında yeni imajın sürüm bilgisi, imaj boyutu, başlangıç adresi, CRC değeri ve geçerlilik durumu tutulabilir. Böylece boot karar mekanizması reset sonrasında bu bilgileri okuyarak yeni imajın çalıştırılıp çalıştırılmayacağına karar verebilir.

Derleme çıktılarından elde edilen boyutlara göre bu projedeki iki firmware imajı aynı anda flash üzerinde saklanabilmektedir. `new-firmware` için ayrılan `0x00000000 - 0x0002FFFF` aralığında 73928 byte'lık imajdan sonra 122680 byte boş alan kalmaktadır. `old-firmware` için ayrılan `0x00030000 - 0x00051FFF` aralığında ise 74324 byte'lık imajdan sonra 64940 byte boş alan kalmaktadır. Bu nedenle şartnamede belirtilen cihaz diskine/data alanına kaydetme fikri, bu raporda flash üzerinde ayrılmış staging bölgesine yeni imajın veri olarak yerleştirilmesi şeklinde yorumlanmıştır.

Bileşen	Görevi	Bu Projedeki Kullanımı
`old-firmware`	Mevcut çalışan firmware	Aktif uygulama alanında tutulur
`new-firmware`	Aday yeni firmware	Staging alanında veri olarak saklanır
Metadata	İmaj hakkında doğrulama bilgisi tutar	Sürüm, boyut, CRC ve geçerlilik bilgisi içerir
BIM / Bootloader	Boot sırasında imaj seçimi yapar	Geçerli yeni imaj varsa onu, yoksa eski imajı seçer
CCFG	Cihaz konfigürasyon bilgilerini tutar	Korunması gereken kritik alan olarak değerlendirilir

Önerilen aktivasyon süreci şu adımlardan oluşmaktadır:

1. Cihaza `old-firmware` aktif uygulama olarak yüklenir.
2. `new-firmware` imajı flash bellekte ayrılmış staging alanına veri olarak yerleştirilir.
3. Yeni imajın boyutu ve başlangıç adresi kontrol edilir.
4. Yeni imaj için CRC veya benzeri bütünlük kontrolü hesaplanır.
5. Metadata alanına yeni imajın geçerli olduğunu gösteren bilgi yazılır.
6. Cihaz resetlenir.
7. Reset sonrası BIM/bootloader metadata alanını kontrol eder.
8. Yeni imaj geçerliyse `new-firmware` seçilir.
9. Yeni imaj geçersizse sistem `old-firmware` ile çalışmaya devam eder.

Yukarıdaki aktivasyon akışı, ikinci firmware imajının doğrudan çalıştırılmadığını; önce staging alanında saklandığını, ardından doğrulama ve metadata güncellemesiyle boot sürecine aday hale getirildiğini göstermektedir. Bu yaklaşım, hatalı bir firmware imajının cihazı kullanılamaz hale getirmesini önlemek için tercih edilmiştir.

Bu tasarımda en önemli nokta, `new-firmware` imajının flash bellekte bulunmasının tek başına yeterli olmamasıdır. Bir imajın boot edilebilir hale gelmesi için başlangıç adresinin, vektör tablosunun, imaj başlığının ve bütünlük bilgisinin boot mekanizması tarafından tanınabilir olması gerekir. Bu nedenle ikinci firmware imajı, yalnızca uygulama koduyla aktif hale getirilemez. Uygulama kodu çalışırken yeni imajı bir bölgeye yazabilir; fakat reset sonrasında hangi imajın çalıştırılacağına karar verme görevi bootloader/BIM tarafındadır.

Bu yaklaşım aynı zamanda fallback imkanı sağlar. Eğer yeni firmware eksik, bozuk veya uyumsuz ise sistemin mevcut `old-firmware` ile çalışmaya devam etmesi mümkündür. Bu nedenle güvenli firmware

güncelleme tasarımında staging alanı, metadata kontrolü ve bootloader kararı birlikte düşünülmelidir.

10 Riskler ve Sınırlılıklar

Bu çalışma, CC1352R platformunda ikinci bir firmware imajının flash bellekte nasıl konumlandırılabileceğini ve bu imajın boot süreciyle nasıl ilişkilendirilebileceğini incelemektedir. Ancak çalışma, üretim seviyesinde tam güvenli bir OTA firmware güncelleme sistemi olarak değerlendirilmemelidir. Bunun temel nedeni, gerçek bir firmware değişim sürecinin yalnızca yeni imajı belleğe yazmakla tamamlanmamasıdır. Yeni imajın geçerli olduğunun doğrulanması, boot zinciri tarafından tanınması, hatalı güncelleme durumunda eski imaja geri dönülebilmesi ve kritik bellek bölgelerinin korunması gerekir (*Boot Image Manager*, t.y.; *Over-the-Air Download Overview for CC13xx and CC26xx*, t.y.).

Bu kapsamda çalışmanın başlıca riskleri ve sınırlılıkları aşağıda verilmiştir.

Risk / Sınırlılık	Açıklama	Etkisi
Bootloader/BIM entegrasyonunun sınırlı olması	Yeni firmware flash üzerinde saklansa bile boot mekanizması tarafından seçilmezse çalıştırılamaz	Yeni imajın otomatik başlaması mümkün olmaz
Flash alanının sınırlı olması	Aynı anda iki tam firmware imajı saklamak her zaman mümkün olmayabilir	Staging alanı boyutu dikkatle planlanmalıdır
CCFG alanının bozulma riski	Yanlış flash silme/yazma işlemi CCFG bölgesini etkileyebilir	Cihazın boot veya debug davranışı bozulabilir
Çalışan imajın üzerine yazma riski	Aktif firmware alanı yanlışlıkla silinirse cihaz açılmayabilir	Güvenli fallback kaybedilir
Metadata eksikliği	Yeni imajın sürüm, boyut ve CRC bilgileri tutulmazsa boot kararı güvenilir olmaz	Geçersiz imaj çalıştırılabilir
Güç kesintisi riski	Flash yazma sırasında güç kesilirse imaj eksik kalabilir	Güncelleme yarıda kalabilir
Kriptografik doğrulama eksikliği	CRC yalnızca hata tespiti sağlar, imajın kaynağını doğrulamaz	Yetkisiz veya değiştirilmiş imaj riski oluşur
Tam rollback mekanizmasının olmaması	Yeni imaj başarısız olursa otomatik geri dönüş yapılamayabilir	Sistem kurtarma kabiliyeti sınırlı kalır

Bu riskler nedeniyle önerilen tasarımda yeni firmware imajı doğrudan aktif uygulama alanına yazılmamaktadır. Bunun yerine ayrı bir staging alanında tutulmakta ve doğrulama adımlarından sonra boot sürecine aday hale getirilmektedir. Bu yaklaşım, çalışan `old-firmware` imajını koruduğu için daha güvenli bir model sunar.

Bununla birlikte, yalnızca staging alanı kullanmak tek başına yeterli değildir. Yeni imajın gerçekten çalıştırılabilmesi için BIM veya benzeri bir bootloader yapısının metadata alanını okuyabilmesi, imaj bütünlüğünü kontrol edebilmesi ve doğru başlangıç adresine dallanabilmesi gerekir. Aksi halde `new-firmware` flash bellekte bulunsa bile cihaz reset sonrasında bu imajı çalıştırmaz.

Bir diğer önemli sınırlılık flash belleğin silme/yazma yapısından kaynaklanmaktadır. Flash bellekte yazma işlemi yapılmadan önce ilgili sektörün silinmesi gerekir. Bu nedenle yanlış adres aralığında yapılacak bir silme işlemi, çalışan uygulama imajını, metadata alanını, BIM bölgesini veya CCFG gibi kritik alanları bozabilir. Kaynak kısıtlı IoT cihazlarında firmware güncelleme süreci; bellek, enerji tüketimi, flash yıpranması ve güvenlik gibi sınırlamalarla birlikte ele alınmalıdır (Arakadakis vd. 2020). OTAP/OAD literatüründe de bu kısıtlar firmware güncelleme tasarımında önemli başlıklar olarak ele alınmaktadır (*OAD Application*, t.y.; *UniFlash*, t.y.).

Bu çalışma kapsamında kriptografik imza doğrulama, güvenli anahtar yönetimi, tam rollback, otomatik recovery ve üretim seviyesinde hata kurtarma mekanizmaları ayrıntılı olarak gerçeklenmemiştir. Bunlar, sistemin daha güvenli ve sahada kullanılabilir hale getirilebilmesi için gelecek çalışma başlıkları olarak değerlendirilmelidir.

11 Gelecek Çalışmalar

Bu çalışma, CC1352R platformunda ikinci firmware imajının flash bellekte nasıl konumlandırılabileceğini ve boot süreciyle nasıl ilişkilendirilebileceğini incelemiştir. Ancak sistemin üretim seviyesinde güvenli, dayanıklı ve tamamen otomatik bir firmware güncelleme mekanizmasına dönüşebilmesi için bazı ek geliştirmelere ihtiyaç vardır.

İlk olarak, gerçek bir BIM/bootloader entegrasyonu yapılmalıdır. Bu çalışmada `new-firmware` imajının staging alanında saklanması ve metadata ile doğrulanması önerilmiştir. Fakat bu imajın reset sonrasında otomatik olarak seçilebilmesi için bootloader tarafında imaj başlığı, sürüm bilgisi, CRC değeri ve başlangıç adresi gibi alanların okunması gerekir. Bu nedenle gelecek aşamada BIM'in aday imajı tanıyacak ve uygun durumda bu imaja dallanacak şekilde yapılandırılması hedeflenebilir.

İkinci olarak, yalnızca CRC tabanlı bütünlük kontrolü yeterli görülmemelidir. CRC, aktarım veya yazma sırasında oluşan bozulmaları tespit etmek için faydalıdır; ancak firmware imajının gerçekten güvenilir bir kaynak tarafından üretildiğini kanıtlamaz. Bu nedenle ileride firmware imajları dijital imza ile doğrulanabilir. Böylece cihaz yalnızca yetkili kişi veya sistem tarafından imzalanmış firmware'i kabul eder.

Üçüncü olarak, A/B firmware slot yapısı değerlendirilebilir. Bu yaklaşımda cihazda iki ayrı uygulama alanı bulunur. Bir alan aktif firmware'i çalıştırırken diğer alan yeni firmware için kullanılır. Yeni imaj başarılı şekilde doğrulanırsa boot mekanizması yeni slota geçer. Eğer yeni imaj başarısız olursa cihaz eski slottaki sağlam firmware'e geri dönebilir. Bu yapı, sahada çalışan cihazlar için daha güvenli bir güncelleme modeli sunar.

Dördüncü olarak, rollback mekanizması geliştirilebilir. Yeni firmware ilk açılışta hata verirse veya beklenen sağlık kontrolünü geçemezse sistem otomatik olarak eski firmware'e dönebilmelidir. Bunun için metadata alanında yalnızca "geçerli imaj" bilgisi değil, aynı zamanda "test ediliyor", "başarılı açıldı" veya "geri dön" gibi durum bilgileri de tutulabilir.

Beşinci olarak, UniFlash ile yapılan yükleme süreci daha kontrollü hale getirilebilir. Özellikle CCFG ve BIM gibi kritik alanların yanlışlıkla silinmemesi için flashlama öncesi adres sınırlarını kontrol eden yardımcı betikler kullanılabilir. Bu betikler, yeni firmware boyutunun staging alanını aşıp aşmadığını ve CCFG bölgesiyle çakışma olup olmadığını otomatik olarak denetleyebilir.

Altıncı olarak, bu çalışma ileride gerçek OTA aktarım mekanizmasıyla birleştirilebilir. Bu raporun kapsamı üçüncü iş parçacığı olduğu için yeni firmware'in kablosuz ortamda gönderilmesi ana hedef değildir. Ancak sonraki aşamada, staging alanına yazılacak `new-firmware` imajı kablosuz ağ üzerinden parça parça alınabilir. Böyle bir sistemde blok numarası, CRC, eksik parça takibi, yeniden iletim ve aktarım sonunda bütün imaj doğrulaması gibi mekanizmalar eklenmelidir.

Son olarak, enerji tüketimi ve flash ömrü daha ayrıntılı incelenebilir. Flash bellekte her yazma işlemi öncesinde silme işlemi gerektiği için gereksiz tekrar yazmalar hem enerji tüketimini artırır hem de flash hücrelerinin kullanım ömrünü azaltabilir. Bu nedenle üretim seviyesinde bir sistemde imaj sıkıştırma, parça bazlı doğrulama, yalnızca değişen blokların yazılması ve güncelleme sırasında enerji durumunun kontrol edilmesi gibi optimizasyonlar değerlendirilebilir.

12 Sonuç

Bu raporda CC1352R üzerinde iki firmware imajının aynı flash içinde planlanması, derleme çıktılarından bellek kullanımının çıkarılması ve boot zincirinin firmware aktivasyonu üzerindeki rolü incelenmiştir. `old-firmware` ve `new-firmware` için alınan `size`, `objdump`, `readelf` ve `nm` çıktıları, imajların yalnızca ham binary dosyalar olmadığını; farklı flash/RAM section'larından oluşan çalıştırılabilir ELF imajları olduğunu göstermektedir.

Elde edilen sonuçlara göre `new-firmware` page 0 user image slotuna, `old-firmware` ise persistent fallback slotuna sığmaktadır. BIM/CCFG alanı uygulama imajlarından ayrılmış ve korunması gereken özel bölge olarak değerlendirilmiştir. Güvenli firmware geçişi için temel öneri, imajları doğru OAD header ve linker yerleşimiyle üretmek, HEX tabanlı programlama kullanmak ve full chip erase işleminden kaçınmaktır.

13 Ekler

13.1 Ek A — `old-firmware` Size Çıktısı

```
   text    data     bss     dec     hex filename
  72672    1452   14520   88644   15a44 build/simplelink/sensortag/cc1352r1/old-firmware.sim
```

13.2 Ek B — `new-firmware` Size Çıktısı

```
   text    data     bss     dec     hex filename
  72276    1452   14520   88248   158b8 build/simplelink/sensortag/cc1352r1/new-firmware.sim
```

13.3 Ek C — Section Analizi Çıktıları

`old-firmware` için seçili section çıktıları:

```
Idx Name          Size      VMA       LMA       File off  Algn
  0 .image_header 00000038  00030000  00030000  00010000  2**0
  1 .resetVecs    00000040  00030100  00030100  00010100  2**2
 10 .text         00011b60  00030140  00030140  00010140  2**2
 11 .ARM.exidx    00000008  00041ca0  00041ca0  00021ca0  2**2
 12 .data         000004d4  20001b20  00041ca8  00031b20  2**2
 13 vtable_ram    000000d8  20002000  0004217c  00032000  2**8
 14 .bss          00003234  200020d8  200020d8  00041830  2**3
 15 .stack        00000604  2000530c  2000530c  00041830  2**0
```

`new-firmware` için seçili section çıktıları:

```
Idx Name          Size      VMA       LMA       File off  Algn
  0 .image_header 00000038  00000000  00000000  00010000  2**0
  1 .resetVecs    00000040  00000100  00000100  00010100  2**2
 10 .text         000119d4  00000140  00000140  00010140  2**2
 11 .ARM.exidx    00000008  00011b14  00011b14  00021b14  2**2
 12 .data         000004d4  20001b20  00011b1c  00021b20  2**2
 13 vtable_ram    000000d8  20002000  00011ff0  00022000  2**8
 14 .bss          00003234  200020d8  200020d8  00031830  2**3
 15 .stack        00000604  2000530c  2000530c  00031830  2**0
```

13.4 Ek D — ELF Header Özetleri

```
old-firmware:
  Class:                             ELF32
  Type:                              EXEC (Executable file)
  Machine:                           ARM
  Entry point address:               0x36d41

new-firmware:
  Class:                             ELF32
  Type:                              EXEC (Executable file)
  Machine:                           ARM
  Entry point address:               0x6cad
```

13.5 Ek E — Ham Çıktı Dosyaları

Raporun teknik kanıtları aşağıdaki klasörde korunmaktadır:

```
docs/report/outputs/
   build-files.txt
   firmware-files.txt
   upload-files.txt
   old-size.txt
   new-size.txt
   old-sections.txt
   new-sections.txt
   old-readelf-header.txt
   new-readelf-header.txt
   old-symbols.txt
   new-symbols.txt
```

Kaynakça

Arakadakis, Konstantinos, Pavlos Charalampidis, Antonis Makrogiannakis, ve Alexandros Fragkiadakis. 2020. *Firmware over-the-air programming techniques for IoT networks - A survey*.

Boot Image Manager. t.y. Texas Instruments. Erişim 16 Mayıs 2026. https://software-dl.ti.com/simplelink/esd/simplelink_cc13xx_cc26xx_sdk/latest/exports/docs/ble5stack/ble_user_guide/html/oad-secure/bim.html.

CC13xx/CC26xx Bootloader and Customer Configuration. t.y. Texas Instruments. Erişim 16 Mayıs 2026. https://www.ti.com/lit/an/swra466e/swra466e.pdf.

Contiki-NG Documentation. t.y. Online Documentation. Erişim 16 Mayıs 2026. https://docs.contiki-ng.org/.

Contiki-NG SimpleLink Platform. t.y. Contiki-NG. Erişim 16 Mayıs 2026. https://docs.contiki-ng.org/en/master/doc/platforms/simplelink.html.

Engin, Yasin, ve El-Hâris Halef. 2026. *Rehydrator Firmware Update Repository*. GitHub Repository. https://github.com/YasinEnginn/rehydrator.

Instruments, Texas. t.y.-a. *CC1352R SimpleLink High-Performance Multi-Band Wireless MCU Datasheet*. Technical Documentation. Erişim 16 Mayıs 2026. https://www.ti.com/lit/ds/symlink/cc1352r.pdf.

Instruments, Texas. t.y.-b. *CC13x2, CC26x2 SimpleLink Wireless MCU Technical Reference Manual*. Technical Reference Manual. Erişim 16 Mayıs 2026. https://www.ti.com/lit/ug/swcu185g/swcu185g.pdf.

OAD Application. t.y. Texas Instruments. Erişim 16 Mayıs 2026. https://software-dl.ti.com/simplelink/esd/simplelink_cc13xx_cc26xx_sdk/8.31.00.11/exports/docs/ble5stack/ble_user_guide/html/ble-stack-oad/oad-application.html.

Over-the-Air Download Overview for CC13xx and CC26xx. t.y. Texas Instruments. Erişim 16 Mayıs 2026. https://software-dl.ti.com/simplelink/esd/simplelink_cc13xx_cc26xx_sdk/6.10.00.29/exports/docs/ble5stack/ble_user_guide/html/ble-stack-5.x-guide/ble5-oad-index-cc13xx_cc26xx.html.

UniFlash. t.y. Texas Instruments. Erişim 16 Mayıs 2026. https://www.ti.com/tool/UNIFLASH.